RELAZIONE TECNICA

Pratica extra: TCP, HTTP e analisi di Slowloris

Laboratorio didattico con Kali Linux e Metasploitable

Nota di correttezza: sono state eseguite verifiche TCP/HTTP e analisi del server. Il repository Slowloris è stato scaricato e ispezionato, ma il programma non è stato avviato. Non si dichiara quindi che sia stato eseguito un attacco DoS né che il servizio sia stato reso indisponibile.

Obiettivo

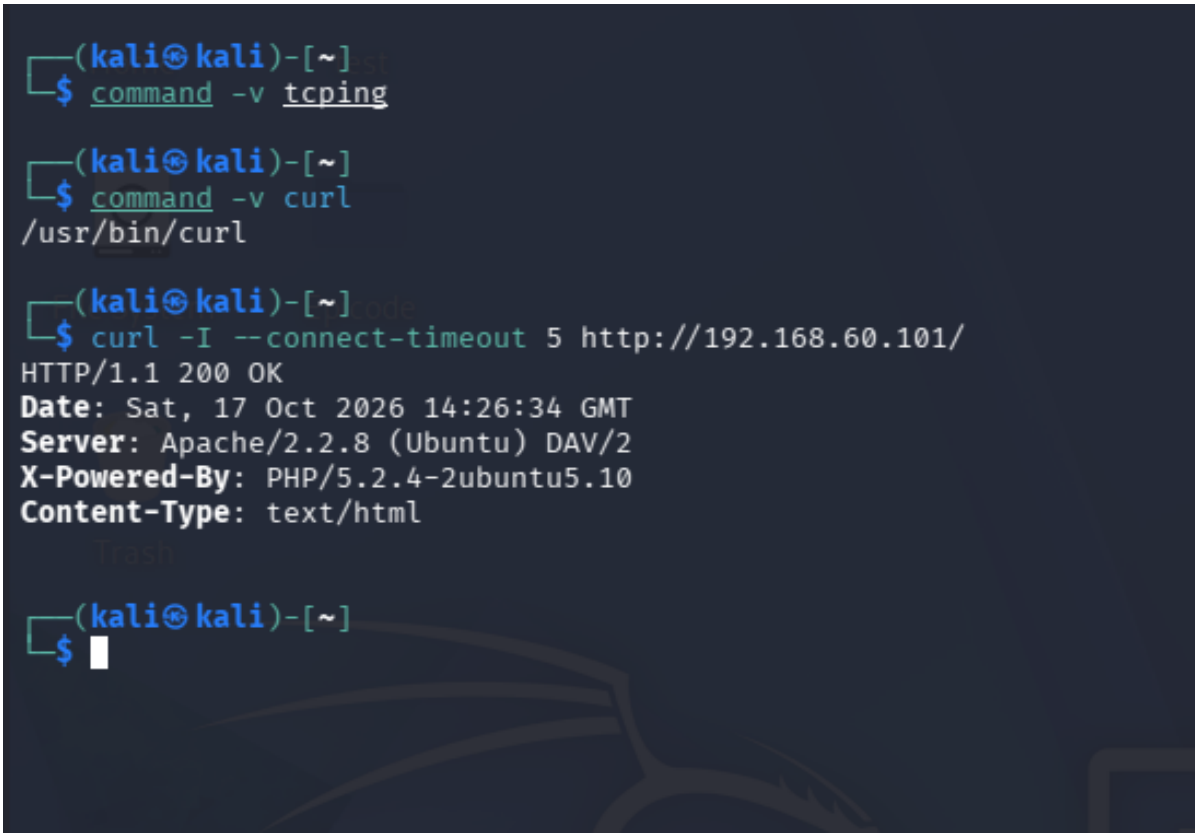
Verificare la connettività TCP verso la porta 80 di Metasploitable, controllare le risposte HTTP, osservare lo stato di Apache e analizzare Slowloris. La relazione distingue i risultati pratici dalla spiegazione teorica.

Ambiente e strumenti

- Kali Linux: 192.168.50.100.
- Metasploitable: 192.168.60.101.
- curl per richieste HTTP HEAD e misurazione del tempo di connessione.
- Netcat (nc) per verificare l'apertura TCP della porta 80.
- Nping per cinque sonde TCP SYN come alternativa a tcping.
- netstat, ps e watch per osservare stato e processi Apache.
- access.log per confermare le richieste HTTP ricevute.
- Git e Python per scaricare e ispezionare Slowloris.

Verifiche TCP e HTTP

La prima richiesta curl ha restituito HTTP/1.1 200 OK. Gli header mostravano Apache/2.2.8 su Ubuntu e PHP/5.2.4-2ubuntu5.10, come apparso nel laboratorio.



Prima verifica HTTP: HTTP/1.1 200 OK.

Netcat ha indicato la porta 80 come open. Il messaggio inverse host lookup failed riguarda la mancata risoluzione DNS inversa e non ha impedito la connessione.



Netcat conferma che la porta TCP 80 è aperta.

Cinque controlli TCP con Netcat hanno restituito exit code 0. Cinque richieste HTTP successive hanno restituito HTTP 200; i tempi di connessione erano nell'ordine di circa 1–2 millisecondi.

```
┌──(kali㉿kali)-[~]
└─$ for i in 1 2 3 4 5; do
  date '+%H:%M:%S'
  nc -z -w 2 192.168.60.101 80
  echo "Exit code: $?"
  sleep 1
done
10:43:06
Exit code: 0
10:43:07
Exit code: 0
10:43:08
Exit code: 0
10:43:09
Exit code: 0
10:43:10
Exit code: 0

┌──(kali㉿kali)-[~]
└─$ for i in 1 2 3 4 5; do
  date '+%H:%M:%S'
  curl -sS -o /dev/null -I --connect-timeout 2 \
    -w 'HTTP status: %{http_code} | Connect time: %{time_connect}s\n' \
    http://192.168.60.101/
  sleep 1
done
10:43:24
HTTP status: 200 | Connect time: 0.001382s
10:43:25
HTTP status: 200 | Connect time: 0.001347s
10:43:26
HTTP status: 200 | Connect time: 0.001679s
10:43:27
HTTP status: 200 | Connect time: 0.001187s
10:43:28
HTTP status: 200 | Connect time: 0.001212s

┌──(kali㉿kali)-[~]
└─$
```

Cinque controlli TCP e cinque risposte HTTP 200.

Il pacchetto tcping non era disponibile nei repository configurati. È stato usato Nping come alternativa; tcping non è stato installato né eseguito.

```
┌──(kali㉿kali)-[~]
└─$ sudo nping --tcp -p 80 --flags syn --count 5 --delay 1s 192.168.60.101
[sudo] password for kali:

Starting Nping 0.7.99 ( https://nmap.org/nping ) at 2026-10-17 10:50 EDT
SENT (0.0173s) TCP 192.168.50.100:56809 > 192.168.60.101:80 S ttl=64 id=60477 iplen=40  seq=1690167155 win=1480
RCVD (0.0223s) TCP 192.168.60.101:80 > 192.168.50.100:56809 SA ttl=63 id=0 iplen=44  seq=2815740308 win=5840 <mss 1
460>
SENT (1.0275s) TCP 192.168.50.100:56809 > 192.168.60.101:80 S ttl=64 id=60477 iplen=40  seq=1690167155 win=1480
RCVD (1.0286s) TCP 192.168.60.101:80 > 192.168.50.100:56809 SA ttl=63 id=0 iplen=44  seq=2831531462 win=5840 <mss 1
460>
SENT (2.3778s) TCP 192.168.50.100:56809 > 192.168.60.101:80 S ttl=64 id=60477 iplen=40  seq=1690167155 win=1480
RCVD (2.3790s) TCP 192.168.60.101:80 > 192.168.50.100:56809 SA ttl=63 id=0 iplen=44  seq=2852647499 win=5840 <mss 1
460>
SENT (3.3904s) TCP 192.168.50.100:56809 > 192.168.60.101:80 S ttl=64 id=60477 iplen=40  seq=1690167155 win=1480
RCVD (3.3915s) TCP 192.168.60.101:80 > 192.168.50.100:56809 SA ttl=63 id=0 iplen=44  seq=2868484784 win=5840 <mss 1
460>
SENT (4.3915s) TCP 192.168.50.100:56809 > 192.168.60.101:80 S ttl=64 id=60477 iplen=40  seq=1690167155 win=1480
RCVD (4.3927s) TCP 192.168.60.101:80 > 192.168.50.100:56809 SA ttl=63 id=0 iplen=44  seq=2884139824 win=5840 <mss 1
460>

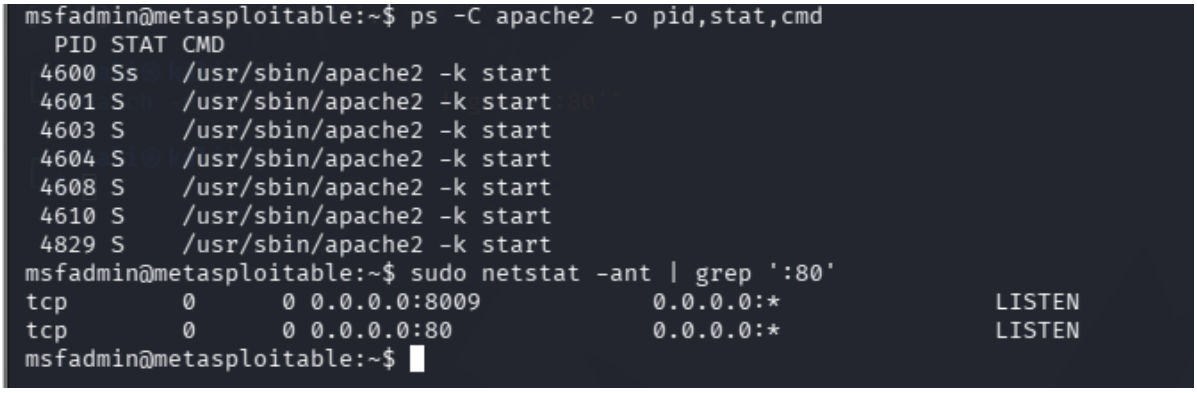
Max rtt: 4.862ms | Min rtt: 1.030ms | Avg rtt: 1.851ms
Raw packets sent: 5 (270B) | Rcvd: 5 (230B) | Lost: 0 (0.00%)
Nping done: 1 IP address pinged in 4.45 seconds

┌──(kali㉿kali)-[~]
└─$
```

Nping: cinque sonde TCP SYN e cinque risposte SYN-ACK.

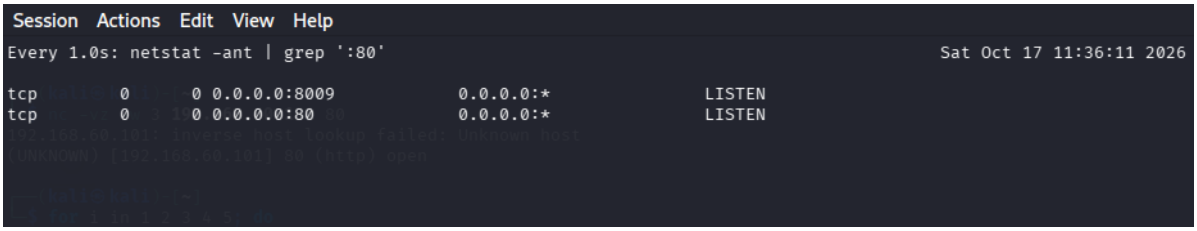
Stato di Apache e monitoraggio

Su Metasploitable, ps ha mostrato il processo principale Apache e diversi processi di lavoro. netstat ha mostrato 0.0.0.0:80 nello stato LISTEN. È stata osservata anche la porta 8009 in ascolto, annotata come informazione secondaria.



Processi Apache e porte 80 e 8009 in ascolto.

watch è stato avviato dopo aver corretto un problema del tipo di terminale xterm-256color. La schermata acquisita mostra la porta 80 in LISTEN. Le richieste HTTP erano brevi e non sempre comparivano durante l'aggiornamento del monitor.



watch aggiorna la visualizzazione dello stato della porta 80.

Il log access.log ha registrato richieste HEAD / HTTP/1.1 provenienti da 192.168.50.100, con codice HTTP 200 e user-agent curl/8.21.0. Questo conferma che le richieste sono arrivate al server e sono state gestite.

```
msfadmin@metasploitable:~$ sudo tail -f /var/log/apache2/access.log
[sudo] password for msfadmin:
Sorry, try again.
[sudo] password for msfadmin:
Sorry, try again.
[sudo] password for msfadmin:
192.168.50.100 - - [17/Oct/2026:11:50:09 -0400] "HEAD / HTTP/1.1" 200 - "-" "curl/8.21.0"
192.168.50.100 - - [17/Oct/2026:11:50:10 -0400] "HEAD / HTTP/1.1" 200 - "-" "curl/8.21.0"
192.168.50.100 - - [17/Oct/2026:11:50:11 -0400] "HEAD / HTTP/1.1" 200 - "-" "curl/8.21.0"
192.168.50.100 - - [17/Oct/2026:11:50:12 -0400] "HEAD / HTTP/1.1" 200 - "-" "curl/8.21.0"
192.168.50.100 - - [17/Oct/2026:11:50:14 -0400] "HEAD / HTTP/1.1" 200 - "-" "curl/8.21.0"
192.168.50.100 - - [17/Oct/2026:11:53:20 -0400] "HEAD / HTTP/1.1" 200 - "-" "curl/8.21.0"
192.168.50.100 - - [17/Oct/2026:11:53:21 -0400] "HEAD / HTTP/1.1" 200 - "-" "curl/8.21.0"
192.168.50.100 - - [17/Oct/2026:11:53:22 -0400] "HEAD / HTTP/1.1" 200 - "-" "curl/8.21.0"
192.168.50.100 - - [17/Oct/2026:11:53:24 -0400] "HEAD / HTTP/1.1" 200 - "-" "curl/8.21.0"
192.168.50.100 - - [17/Oct/2026:11:53:25 -0400] "HEAD / HTTP/1.1" 200 - "-" "curl/8.21.0"

msfadmin@metasploitable:~$
```

Log Apache: richieste HEAD da Kali registrate con risposta 200.

Analisi del repository Slowloris

Il repository indicato nella consegna è stato scaricato con Git nella cartella ~/slowloris. È stato individuato slowloris.py e python3 slowloris.py --help ha mostrato opzioni come host, porta, numero di socket, HTTPS e intervallo tra gli invii degli header.

```
┌──(kali㉿kali)-[~]
└─$ cd ~
git clone https://github.com/gkbrk/slowloris.git
Cloning into 'slowloris'...
remote: Enumerating objects: 152, done.
remote: Counting objects: 100% (66/66), done.
remote: Compressing objects: 100% (29/29), done.
remote: Total 152 (delta 39), reused 37 (delta 37), pack-reused 86 (from 2)
Receiving objects: 100% (152/152), 27.79 KiB | 12.00 KiB/s, done.
Resolving deltas: 100% (78/78), done.

┌──(kali㉿kali)-[~]
└─$ cd

┌──(kali㉿kali)-[~]
└─$ cd ~/slowloris
ls -la
total 36
drwxrwxr-x  3 kali kali 4096 Oct 17 10:58 .
drwx------ 33 kali kali 4096 Oct 17 10:58 ..
drwxrwxr-x  7 kali kali 4096 Oct 17 10:58 .git
-rw-rw-r--  1 kali kali 1065 Oct 17 10:58 LICENSE
-rw-rw-r--  1 kali kali   26 Oct 17 10:58 MANIFEST.in
-rw-rw-r--  1 kali kali 2551 Oct 17 10:58 README.md
-rw-rw-r--  1 kali kali  438 Oct 17 10:58 setup.py
-rwxrwxr-x  1 kali kali 8047 Oct 17 10:58 slowloris.py

┌──(kali㉿kali)-[~/slowloris]
└─$ sed -n '1,180p' README.md
# slowloris.py - Simple slowloris in Python

## What is Slowloris?
Slowloris is basically an HTTP Denial of Service attack that affects threaded servers. It works like this:

1. We start making lots of HTTP requests.
2. We send headers periodically (every ~15 seconds) to keep the connections open.
3. We never close the connection unless the server does so. If the server closes a connection, we create a new one
keep doing the same thing.

This exhausts the servers thread pool and the server can't reply to other people.

## Citation

If you found this work useful, please cite it as

```bibtex
@article{gkbrkslowloris,
  title = "Slowloris",
  author = "Gokberk Yaltirakli",
  journal = "github.com",
  year = "2015",
```
```

Download del repository e lettura del README.



```
┌──(kali㉿kali)-[~/slowloris]
└─$ python3 slowloris.py --help
usage: slowloris.py [-h] [-p PORT] [-s SOCKETS] [-v] [-ua] [-x] [--proxy-host PROXY_HOST]
                    [--proxy-port PROXY_PORT] [--https] [--sleeptime SLEEPTIME]
                    [host]

Slowloris, low bandwidth stress test tool for websites

positional arguments:
  host                  Host to perform stress test on

options:
  -h, --help            show this help message and exit
  -p, --port PORT       Port of webserver, usually 80
  -s, --sockets SOCKETS
                        Number of sockets to use in the test
  -v, --verbose         Increases logging
  -ua, --randuseragents
                        Randomizes user-agents with each request
  -x, --useproxy        Use a SOCKS5 proxy for connecting
  --proxy-host PROXY_HOST
                        SOCKS5 proxy host
  --proxy-port PROXY_PORT
                        SOCKS5 proxy port
  --https               Use HTTPS for the requests
  --sleeptime SLEEPTIME
                        Time to sleep between each header sent.

┌──(kali㉿kali)-[~/slowloris]
└─$ grep -nE 'argparse|OptionParser|socket|connect|send|sleep|timeout' slowloris.py
2:import argparse
5:import socket
9:parser = argparse.ArgumentParser(
18:    "--sockets",
20:    help="Number of sockets to use in the test",
42:    help="Use a SOCKS5 proxy for connecting",
57:    "--sleeptime",
58:    dest="sleeptime",
61:    help="Time to sleep between each header sent.",
80:    # and monkey patches socket.socket to connect over
88:        socket.socket = socks.socksocket
89:        logging.info("Using SOCKS5 proxy for connecting...")
101:def send_line(self, line):
103:    self.send(line.encode("utf-8"))
106:def send_header(self, name, value):
107:    self.send_line(f"{name}: {value}")
114:    setattr(ssl.SSLSocket, "send_line", send_line)
115:    setattr(ssl.SSLSocket, "send_header", send_header)
117:list_of_sockets = []
146:setattr(socket.socket, "send_line", send_line)
147:setattr(socket.socket, "send_header", send_header)
150:def init_socket(ip: str):
```

Opzioni mostrate da --help e ispezione del codice.

Slowloris è una tecnica di DoS a livello applicativo: tenta di mantenere aperte connessioni HTTP inviando richieste o header incompleti e aggiornandoli lentamente, così da occupare risorse del server. In questa sessione il codice è stato ispezionato, ma non eseguito.

Differenze tra TCP, HTTP, DoS, DDoS e Slowloris

TCP e HTTP

TCP è il protocollo di trasporto. Un test TCP verifica se è possibile raggiungere una porta e stabilire una connessione. HTTP è il protocollo applicativo che trasporta richieste e risposte. Una porta aperta non garantisce da sola che l'applicazione funzioni: per questo sono stati usati sia Netcat/Nping sia curl.

DoS

Un Denial of Service tenta di rendere un servizio indisponibile o degradarne le prestazioni consumando banda, connessioni, memoria o capacità di elaborazione. Può provenire da una singola sorgente.

DDoS

Un Distributed Denial of Service persegue lo stesso obiettivo da più sorgenti distribuite. La molteplicità delle origini rende più complesso filtrare il traffico basandosi su un solo indirizzo.

Slowloris

Slowloris è una tecnica di DoS HTTP che prova a trattenere connessioni aperte inviando header lentamente o lasciando richieste incomplete. Può influire su server con risorse o thread limitati, senza richiedere necessariamente un grande volume di banda.

Indicatori e contromisure

Indicatori da monitorare:

- Aumento anomalo delle connessioni simultanee o che rimangono aperte a lungo.
- Richieste incomplete e tempi di completamento degli header insolitamente elevati.
- Incremento degli stati TCP, dei processi/thread occupati e dei tempi di risposta HTTP.
- Timeout, errori di disponibilità o anomalie nei log del server web.

Contromisure possibili:

- Configurare timeout appropriati per la ricezione degli header e delle richieste.

- Impostare limiti ragionevoli alle connessioni simultanee e alle risorse per client.
- Utilizzare reverse proxy o WAF con protezioni per richieste lente o incomplete.
- Monitorare connessioni, processi, tempi di risposta e log, con allarmi su anomalie.
- Mantenere aggiornati sistema operativo e server web e verificare le configurazioni in ambiente di test.

Risultati e limiti

Le prove confermano che, durante i controlli, il target era raggiungibile sulla porta TCP 80, Apache era in ascolto e le richieste HTTP HEAD restituivano 200. Nping ha ricevuto cinque risposte SYN-ACK su cinque sonde. I log Apache hanno registrato le richieste provenienti da Kali.

Non è stata eseguita una prova di stress con Slowloris e non è stato osservato un effetto DoS. Non è quindi possibile concludere come il target reagirebbe sotto attacco. Il requisito specifico tcping è stato coperto solo parzialmente con Nping, perché il pacchetto non era disponibile nei repository.

Conclusione personale

Con questa pratica ho imparato a distinguere il controllo della connettività TCP dalla verifica della risposta HTTP. Ho usato Netcat e Nping per verificare la porta 80, curl per controllare le risposte e gli strumenti disponibili su Metasploitable per osservare Apache e i log. Ho inoltre analizzato Slowloris e le differenze tra DoS e DDoS. I risultati descrivono il funzionamento normale del laboratorio; la simulazione dell'attacco non è stata eseguita, quindi non attribuisco al test effetti che non ho osservato.

Appendice — Comandi principali

```
curl -I --connect-timeout 5 http://192.168.60.101/

nc -vz -w 3 192.168.60.101 80

sudo nping --tcp -p 80 --flags syn --count 5 --delay 1s 192.168.60.101

ps -C apache2 -o pid,stat,cmd

sudo netstat -ant | grep ':80'
```

```
watch -n 1 "netstat -ant | grep ':80'"

sudo tail -f /var/log/apache2/access.log

python3 slowloris.py --help
```